

```

    ⑤ fibonacciCache[nthNumber] = 1; // Update the cache.
    return 1;
} else {
    // RECURSIVE CASE
    document.write(indentation +
    "Calling fibonacci(" + (nthNumber - 1) + ") (nthNumber - 1).<br />");
    let result = fibonacci(nthNumber - 1, indent + 1);

    document.write(indentation +
    "Calling fibonacci(" + (nthNumber - 2) + ") (nthNumber - 2).<br />");
    result = result + fibonacci(nthNumber - 2, indent + 1);

    document.write(indentation + "Returning " + result + ".<br />");
    ④ fibonacciCache[nthNumber] = result; // Update the cache.
    return result;
}
}

document.write("<pre>");
document.write(fibonacci(10) + "<br />");
⑤ document.write(fibonacci(10) + "<br />");
document.write("</pre>");
</script>

```

---

If you compare the output of this program to the original recursive Fibonacci program in Chapter 2, you'll find it's much shorter. This reflects the massive reduction of computation needed to achieve the same results:

---

```

fibonacci(10) called.
Calling fibonacci(9) (nthNumber - 1).
.fibonacci(9) called.
.Calling fibonacci(8) (nthNumber - 1).
..fibonacci(8) called.
..Calling fibonacci(7) (nthNumber - 1).
--snip--
.....Calling fibonacci(2) (nthNumber - 1).
.....fibonacci(2) called.
.....Base case fibonacci(2) returning 1.
.....Calling fibonacci(1) (nthNumber - 2).
.....fibonacci(1) called.
.....Base case fibonacci(1) returning 1.
Call to fibonacci(3) returning 2.
.....Calling fibonacci(2) (nthNumber - 2).
.....fibonacci(2) called.

```